

claude-windows / 7dc3da51-af38-47b6-83f6-fdda83b60664

Metadata

```
- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\subagents\workflows\wf_3512746c-54e\agent-a96a3ae8662264543.jsonl`
- SHA-256: `3e18113e6d9214fc7a4bf2df300e39d852655179ee2dc0fa14f2c59df72ac168`
- Source modified: `2026-07-06T09:12:51+00:00`
- Imported at: `2026-07-08T16:00:05+00:00`
- project: `wf_3512746c-54e`
- session_id: `7dc3da51-af38-47b6-83f6-fdda83b60664`
```

Transcript

1. user (2026-07-06T09:07:16.619Z)

Reviewing GitHub PR #39 of the repo at C:/Users/avidu/Projects/Telegram (branch feat/track-e-selective-scan -> main, one commit db87cc0).

It implements ADR 0009 Track E: selective source scanning ? /watch, /unwatch, /channels, /watchlist commands +

an ingestion gate in telethon_client.py that skips unwatched sources. Changed files ONLY: command_service.py (new /watch etc + _resolve_source), commands.py (re-export patterns), db.py (set_source_watched, list_account_sources), telethon_client.py (the gate), tests/test_command_service.py (10 tests).

Diff +254/-3. READ docs/adr/0009-selective-channel-scanning.md ? it is the spec this must conform to.

KEY ADR 0009 REQUIREMENTS (quote/verify against the code):

- Line 37-38: "The default is locked as opt-in: DEFAULT 0, so a freshly-onboarded user scans nothing until they opt in."
- Line 44: "...return early if watched is false, before any dedup or filter work."
- Line 130-131: "New channels discovered after the migration default to watched = 0 and must be explicitly watched."
- Line 48-54 (discoverability): lazy-populated rows should be created watched=0 so they appear in /channels for opt-in;
/channels should also upsert candidates from client.get_dialogs().
- Line 64: "/watch <#name|id>" ? resolve by channel name/username OR id.

Ground rules:

- READ actual code (Read/Grep), cite file:line. You MAY run repros with C:/Users/avidu/Projects/Telegram/.venv/Scripts/python.exe (build schema via db._apply_schema on aiosqlite ":memory:"); seed via db.upsert_account + db.upsert_source; drive command_service.CommandService().handle()).
- NOTE: real telegram sources have a NUMERIC provider_source_id (str(channel_id) e.g. '-1001234567890'); upsert_channel creates them with watched=True. The on_new_message gate is inside a '# pragma: no cover' function.
- Authoritative gate ALREADY RUN (don't re-run full suite): ruff check + format clean; pytest 492 passed @ 97.10% (floor 80%).
- Report ONLY defects in THIS PR's diff. Deviation from the ACCEPTED ADR 0009 is a high-value finding. No praise.
Concrete failure scenario per finding, ranked most-severe first, honest severity.
- Severity: blocker (feature broken / data loss on the live path), high (real bug or ADR-contract violation users hit),
medium (edge/latent/half-broken feature), low (hygiene/UX/discoverability).

DIMENSION: command correctness ? /watch, /unwatch, /channels, /watchlist, _resolve_source.
SEED to verify/refute (I believe CONFIRMED, medium): /watch @username is non-functional for REAL sources.

_resolve_source (command_service.py) does target.lstrip('@') then
get_source_by_provider_id(provider_source_id=target),
but real sources have a NUMERIC provider_source_id, never a username ? so '/watch @somechannel'
returns "not found"
even for a known channel. The test test_watch_by_username passes only because it SEEDS an
unrealistic source with
provider_source_id='@mychan'. ADR line 64 says '/watch <#name|id>' (name OR id). Reproduce with
a realistic source
(numeric id, username/display-name in title) and confirm @username + name both fail; only
numeric id works.
Also check: /channels claims (ADR 48-54) it should enumerate from client.get_dialogs() so users
can browse channels
with no traffic yet ? is that implemented, or does /channels only list already-seen sources?
rstrip('@') strips
multiple @. set_source_watched/list_account_sources correctness. Report findings.

2. assistant (2026-07-06T09:07:20.153Z)

I'll start by reading the ADR spec and the changed files.

3. user (2026-07-06T09:07:21.152Z)

```
1      # ADR 0009: Selective channel scanning and multiplexed routing
2
3      - Status: Accepted
4      - Date: 2026-07-04
5      - Decision owner: project owner
6
7      ## Context
8
9      Today the userbot scans every channel/group the account is a member of.
10     `manager.py` registers `events.NewMessage(incoming=True)` with no `chats=`
11     filter, and `on_new_message` processes every video regardless of source. The
12     `channels` table exists but is populated lazily as a record of seen channels;
13     it is not used as a filter.
14
15     The product vision is a multiplexing video re-distribution system:
16
17     - A small set of high-quality source channels (e.g. spiritual, classical
18       music) are scanned for incoming videos.
19     - Matches are routed to themed destination broadcast channels (e.g. a
20       Reggae channel, a Rap channel) that subscribers opt into.
21     - A user therefore subscribes to outputs by topic, not to inputs by source.
22
23     This requires two capabilities the current architecture does not have:
24
25     1. Selective scanning ? choose which source channels are watched, rather
26       than ingesting all of them.
27     2. Per-source ? per-destination routing ? a match from source channel X is
28       delivered to a specific destination channel Y, instead of the single global
29       `delivery_target` (ADR 0005).
30
31     ## Decision
32
33     Add three additive, independently-shippable capabilities:
34
35     ### 1. Selective scanning (watched-flag)
36
37     Add a `watched` flag to the `channels` table. The default is locked as
38     opt-in: DEFAULT 0, so a freshly-onboarded user scans nothing until they
39     explicitly `/watch` a channel. This matches the multiplexing product shape
40     (sources are a curated set) and avoids blindly re-ingesting every chat the
41     account is a member of.
42
43     Gate `on_new_message` at the top: after computing `channel_id`, look up the
```

```

44     row and `return` early if `watched` is false, before any dedup or filter work.
45
46     This is a small migration plus ~5 lines in `telethon_client.py`.
47
48     **Discoverability note.** With `DEFAULT 0` plus the current lazy population
49     (a `channels` row is only inserted when a message is first seen), opt-in is
50     necessary but not sufficient for discoverability: a channel the account has
51     never received a message from has no row to flag. The `/channels` command
52     (step 2 below) therefore upserts candidate rows from `client.get_dialogs()`
53     so users can browse and watch channels they have not yet received traffic
54     from. The flag default and the enumeration flow are orthogonal ? both are
55     required.
56
57     ### 2. Channel enumeration and management commands
58
59     New Saved Messages commands (and, via ADR 0008, bot commands) building on the
60     existing `commands.py` pattern:
61
62     - `/channels` ? list the user's channels (via `client.get_dialogs()`), already
63       used in `backfill.py`) with their watched state.
64     - `/watch <#name|id>` ? set `watched=1` for a source channel.
65     - `/unwatch <#name|id>` ? set `watched=0`.
66     - `/watchlist` ? list only watched sources.
67
68     ### 3. Per-source ? per-destination routing rules
69
70     Introduce a `routing_rules` table (the canonical name; issue #25's `routes`
71     refers to the same concept and should be aligned to `routing_rules`), e.g.:
72
73     ```
74     routing_rules (
75         user_id          INTEGER NOT NULL REFERENCES users(id),
76         source_channel_id INTEGER NOT NULL,
77         dest_channel_id   INTEGER NOT NULL,    -- a broadcast channel / @username
78         filter_config_id  INTEGER,             -- optional per-route filter override
79         PRIMARY KEY (user_id, source_channel_id, dest_channel_id)
80     )
81     ```
82
83     On a match, look up applicable routing rules by `source_channel_id` and
84     deliver to each matching destination. The delivery layer (`delivery.py`)
85     already accepts a `target` per call, so this generalises the existing single
86     `delivery_target` to a per-match lookup. Users with no routing rules fall
87     back to the current global `delivery_target` behaviour (ADR 0005), preserving
88     backwards compatibility.
89
90     ## Consequences
91
92     **Positive**
93     - Enables the multiplexing product shape: scan curated sources, publish to
94       topical broadcast channels, let subscribers pick outputs by interest.
95     - Each capability is additive and independently deployable; none disturbs the
96       existing FORWARD/LINK/BOTH paths or the metadata-first ingestion (ADR 0003).
97     - Dedup (ADR 0006) still holds: Tier 1 (`document_id`) keys the `*original*`
98       file, so a routed re-upload is not re-processed.
99     - Channel enumeration reuses `client.get_dialogs()` already present in
100       `backfill.py`; no new Telethon capability is required.
101
102     **Negative**
103     - New schema (`channels.watched`, `routing_rules`) with migrations.
104     - **Routing loop invariant.** With multiplexed routing, the userbot account
105       may be a member of a `*destination*` broadcast channel and re-see its own
106       output, producing an infinite forward loop. The contract is: **a channel
107       is either a source or a destination, never both.** This is enforced in
108       `on_new_message` by excluding any channel that is a configured destination

```

```

109     (`source_channel_id IN (SELECT dest_channel_id FROM routing_rules WHERE
110     user_id = ?)`) before the watched-flag check. Issue #25 Track E implements
111     this; the invariant is stated here so the handler has a contract to code
112     against.
113 - Per-route filter overrides add config complexity; v1 may ship with global
114   filters only and a single implicit route (`source ? global target`), then
115   layer explicit per-route rules later.
116
117   ## Backwards compatibility
118
119   Adding `channels.watched` with `DEFAULT 0` would silently stop every existing
120   install from receiving matches until the user re-watches their channels ? a
121   behaviour change disguised as a schema change. To preserve the current
122   "ingest everything" semantics for existing installs, the migration backfills:
123
124   ```sql
125   ALTER TABLE channels ADD COLUMN watched INTEGER NOT NULL DEFAULT 0;
126   UPDATE channels SET watched = 1 WHERE seen_first_at IS NOT NULL;
127   ```
128
129   i.e. any channel the install has *already* seen traffic from is implicitly
130   watched, so pre-migration users observe no change. New channels discovered
131   after the migration default to `watched = 0` and must be explicitly watched ?
132   which is the intended new UX. This mirrors the "old rows behave exactly as
133   before" pattern established by ADR 0005.
134
135   ## Rollout order
136
137   1. **Selective scanning** (`channels.watched` + `/watch`, `/unwatch`,
138     `/channels`). Smallest unit; immediately useful even without routing.
139   2. **Named destination channels** ? extend `/set delivery_target` to allow
140     switching among a user-defined set, not just one global value.
141   3. **Per-source ? per-destination routing rules** (`routing_rules` table +
142     `/route <source> <dest>` command). The full multiplexing vision.
143
144   Each step is shippable and useful on its own; step 3 is not required to
145   deliver value from steps 1?2.
146
147   ## Alternatives considered
148
149   - **Server-side `chats=` filter at handler registration:** rejected ? the
150     watched set is per-user and mutable at runtime; re-registering Telethon
151     handlers on every `/watch` change is brittle. An in-handler lookup against
152     the `channels` table is simpler and hot-swappable.
153   - **Content-based classification (auto-detect genre):** explicitly out of
154     scope for v1. Routing is user-defined by source channel; automatic topic
155     detection could be a future enhancement but is not required to deliver the
156     multiplexing UX.
157
158   ## References
159
160   - ADR 0001 (userbot ingestion) ? the capability this selectively constrains.
161   - ADR 0005 (per-user delivery target) ? the single-target delivery this
162     generalises into per-route targets.
163   - ADR 0006 (content dedup) ? still applies; routed re-uploads are keyed by
164     the original `document_id`.
165   - ADR 0008 (bot API onboarding) ? the front-door that exposes `/watch`,
166     `/unwatch`, and routing commands to public users.
167

```

4. user (2026-07-06T09:07:21.347Z)

File does not exist. Note: your current working directory is C:\Users\avidu\Projects\Telegram.

5. assistant (2026-07-06T09:07:23.266Z)

The file doesn't exist at that path. Let me find the actual files.

6. user (2026-07-06T09:07:25.615Z)

Exit code 2

```
db87cc0 feat(scanning): selective source scanning ? /watch, /unwatch, /channels, /watchlist (ADR 0009, Track E)
```

```
35f4f70 feat(commands): extract CommandService ? provider-independent command execution (ADR 0011, Track D) (#38)
```

```
12d21b9 feat(delivery): destinations, routing_rules, delivery_attempts (ADR 0011, Track C) (#37)
```

```
---
src/tg_video_filter/command_service.py | 104 ++++++
src/tg_video_filter/commands.py       |   6 +-
src/tg_video_filter/db.py             |  28 ++++++
src/tg_video_filter/telethon_client.py |   9 +++
tests/test_command_service.py         | 110 ++++++
5 files changed, 254 insertions(+), 3 deletions(-)
```

---files---

ls: cannot access '*.py': No such file or directory

7. assistant (2026-07-06T09:07:29.389Z)

Files are under `src/tg\_video\_filter/`. Let me read the diff and the relevant files.

8. user (2026-07-06T09:07:30.753Z)

```
diff --git a/src/tg_video_filter/command_service.py b/src/tg_video_filter/command_service.py
```

```
index 665c0cb..444ac71 100644
```

```
--- a/src/tg_video_filter/command_service.py
```

```
+++ b/src/tg_video_filter/command_service.py
```

```
@@ -18,7 +18,7 @@ from dataclasses import dataclass
import aiosqlite
```

```
from tg_video_filter import db
-from tg_video_filter.models import FilterConfig
+from tg_video_filter.models import FilterConfig, Source
```

```
log = logging.getLogger(__name__)
```

```
@@ -38,6 +38,13 @@ _EXCLUDE_HELP_RE = re.compile(r"^/set\s+exclude\s+", re.IGNORECASE)
_MIN_RES_HELP_RE = re.compile(r"^/set\s+min\s+(?:resolution|res)\s+", re.IGNORECASE)
```

```
+# --- source scanning regex (ADR 0009, Track E) -----
```

```
+
+_WATCH_RE = re.compile(r"^/watch\s+(.+)$", re.IGNORECASE)
+_UNWATCH_RE = re.compile(r"^/unwatch\s+(.+)$", re.IGNORECASE)
+_CHANNELS_RE = re.compile(r"^/channels$", re.IGNORECASE)
+_WATCHLIST_RE = re.compile(r"^/watchlist$", re.IGNORECASE)
+
```

```
# --- unit parsing -----
```

```
@@ -306,6 +313,22 @@ class CommandService:
    if _SHOW_CONFIG_RE.match(text):
        return CommandResult(format_config(current_cfg))
```

```
+
+    # --- source scanning commands (ADR 0009, Track E) -----
```

```
+
+    m = _WATCH_RE.match(text)
+    if m:
+        return await self._handle_watch(conn, user_id, m.group(1).strip(), current_cfg)
+
+    m = _UNWATCH_RE.match(text)
```

```

+         if m:
+             return await self._handle_unwatch(conn, user_id, m.group(1).strip(), current_cfg)
+
+         if _CHANNELS_RE.match(text):
+             return await self._handle_channels(conn, user_id)
+
+         if _WATCHLIST_RE.match(text):
+             return await self._handle_watchlist(conn, user_id)
+
+         # --- /show commands -----
+         if _SHOW_COMMANDS_RE.match(text):
+             return CommandResult(format_commands())
@@ -313,6 +336,85 @@ class CommandService:
+         # Not a recognised command ? return empty result (noise).
+         return CommandResult()
+
+     # -- source scanning handlers (ADR 0009, Track E) -----
+
+     async def _handle_watch(
+         self, conn: aiomysqlite.Connection, user_id: int, target: str, _cfg: FilterConfig
+     ) -> CommandResult:
+         """Parse `/watch <@username|channel_id>` and toggle `watched=1`."""
+         src = await self._resolve_source(conn, user_id, target)
+         if src is None:
+             return CommandResult(
+                 f"? Source `{target}` not found. Use `/channels` to list "
+                 f"available sources.\n\nSources are discovered as messages "
+                 f"arrive; a channel you haven't received traffic from may not "
+                 f"appear yet."
+             )
+         await db.set_source_watched(conn, user_id, src.id, True) # type: ignore[arg-type]
+         label = src.title or src.provider_source_id
+         return CommandResult(f"? Now watching **{label}** (`{src.provider_source_id}`).")
+
+     async def _handle_unwatch(
+         self, conn: aiomysqlite.Connection, user_id: int, target: str, _cfg: FilterConfig
+     ) -> CommandResult:
+         """Parse `/unwatch <@username|channel_id>` and toggle `watched=0`."""
+         src = await self._resolve_source(conn, user_id, target)
+         if src is None:
+             return CommandResult(
+                 f"? Source `{target}` not found. Use `/channels` to list sources."
+             )
+         await db.set_source_watched(conn, user_id, src.id, False) # type: ignore[arg-type]
+         label = src.title or src.provider_source_id
+         return CommandResult(f"? Stopped watching **{label}** (`{src.provider_source_id}`).")
+
+     async def _handle_channels(self, conn: aiomysqlite.Connection, user_id: int) ->
CommandResult:
+         """List all known sources with watch status."""
+         sources = await db.list_account_sources(conn, user_id)
+         if not sources:
+             return CommandResult(
+                 "? **No sources yet.**\n\n"
+                 "Sources are discovered when a new message arrives from a "
+                 "channel. Run `/backfill` to discover historical channels, "
+                 "or wait for new traffic."
+             )
+         lines = ["? **All sources**", ""]
+         for s in sources:
+             icon = "?" if s.watched else ""
+             label = s.title or s.provider_source_id
+             lines.append(f"{icon} `{s.provider_source_id}` ? {label}")
+         return CommandResult("\n".join(lines))

```

```

+     async def _handle_watchlist(self, conn: aiomysqlite.Connection, user_id: int) ->
CommandResult:
+         """List only watched sources."""
+         sources = await db.list_account_sources(conn, user_id, watched_only=True)
+         if not sources:
+             return CommandResult(
+                 """? **No watched sources.**\n\n"""
+                 "Use `/channels` to see available sources, then `/watch <id>` "
+                 "to start scanning."
+             )
+         lines = ["? **Watched sources**", ""]
+         for s in sources:
+             label = s.title or s.provider_source_id
+             lines.append(f"? {s.provider_source_id} ? {label}")
+         return CommandResult("\n".join(lines))
+
+     async def _resolve_source(
+         self, conn: aiomysqlite.Connection, user_id: int, target: str
+     ) -> Source | None:
+         """Resolve a user-provided source identifier to a ``Source`` row.
+
+         Accepts ``@username`` or a numeric channel id string. The ``@`` prefix
+         is stripped before lookup since Telegram channel ids are numeric.
+         """
+         target = target.rstrip("@")
+         # Try by provider-native id first (the common case).
+         src = await db.get_source_by_provider_id(conn, user_id, "telegram", target)
+         if src is not None:
+             return src
+         # Fall back: try with @ prefix in the target column (just in case).
+         return await db.get_source_by_provider_id(conn, user_id, "telegram", f"@{target}")
+
+     # -- internal -----
+
+     async def _update(
diff --git a/src/tg_video_filter/commands.py b/src/tg_video_filter/commands.py
index 974be99..3aea1f6 100644
--- a/src/tg_video_filter/commands.py
+++ b/src/tg_video_filter/commands.py
@@ -27,7 +27,8 @@ from tg_video_filter.models import FilterConfig, User

log = logging.getLogger(__name__)

-from tg_video_filter.command_service import ( # noqa: E402
+from tg_video_filter.command_service import ( # noqa: E402, F401
+    _CHANNELS_RE,
+    _EXCLUDE_HELP_RE,
+    _EXCLUDE_RE,
+    _LENGTH_RE,
@@ -37,6 +38,9 @@ from tg_video_filter.command_service import ( # noqa: E402
+    _SHOW_COMMANDS_RE,
+    _SHOW_CONFIG_RE,
+    _SIZE_RE,
+    _UNWATCH_RE,
+    _WATCH_RE,
+    _WATCHLIST_RE,
+    _format_duration,
+    _format_size,
+    format_commands,
diff --git a/src/tg_video_filter/db.py b/src/tg_video_filter/db.py
index bfaadec..bee716b 100644
--- a/src/tg_video_filter/db.py
+++ b/src/tg_video_filter/db.py
@@ -1534,6 +1534,32 @@ async def get_source_by_id(conn: aiomysqlite.Connection, source_id: int) ->
Source

```

```

    return _row_to_source(row) if row else None

+async def set_source_watched(
+    conn: aiomysqlite.Connection, account_id: int, source_id: int, watched: bool
+) -> None:
+    """Toggle the ``watched`` flag on a source (ADR 0009, Track E)."""
+    await conn.execute(
+        "UPDATE sources SET watched = ? WHERE id = ? AND account_id = ?",
+        (int(watched), source_id, account_id),
+    )
+    await conn.commit()
+
+
+async def list_account_sources(
+    conn: aiomysqlite.Connection, account_id: int, *, watched_only: bool = False
+) -> list[Source]:
+    """List all sources for an account, optionally filtering to watched only."""
+    conn.row_factory = aiomysqlite.Row
+    query = "SELECT * FROM sources WHERE account_id = ?"
+    params: tuple = (account_id,)
+    if watched_only:
+        query += " AND watched = 1"
+    query += " ORDER BY watched DESC, provider_source_id ASC"
+    async with conn.execute(query, params) as cur:
+        rows = await cur.fetchall()
+    return [_row_to_source(r) for r in rows]
+
+
+async def insert_video(conn: aiomysqlite.Connection, video: VideoRecord) -> VideoRecord:
+    """Deprecated shim: insert into ``content_items`` (ADR 0011).

@@ -1717,6 +1743,8 @@ __all__ = [
    "upsert_source",
    "get_source_by_provider_id",
    "get_source_by_id",
+    "set_source_watched",
+    "list_account_sources",
    "insert_content_item",
    "get_content_item_by_message",
    "get_content_item_by_dedup_key",
diff --git a/src/tg_video_filter/telethon_client.py b/src/tg_video_filter/telethon_client.py
index 883e3f8..ebb7416 100644
--- a/src/tg_video_filter/telethon_client.py
+++ b/src/tg_video_filter/telethon_client.py
@@ -181,6 +181,15 @@ def make_new_message_handler(
    message_id = message.id
    document_id = _extract_document_id(message)

+    # --- watched gate (ADR 0009, Track E) -----
+    # Skip ingestion for unwatched sources. The channel must already be in
+    # sources (lazy-created by the /channels enumeration or the legacy
+    # upsert_channel below); if absent, we let the legacy path upsert it
+    # with watched=True for backwards-compatibility.
+    src = await db.get_source_by_provider_id(conn, user.id, "telegram", str(channel_id))
+    if src is not None and not src.watched:
+        return # silently skip unwatched source
+
    # --- three-tier dedup (cheapest first) ---

    # Tier 0: same (user, channel, message) instance ? already processed.

```

9. assistant (2026-07-06T09:07:35.450Z)

Now let me look at the key supporting code: `get\_source\_by\_provider\_id`, `upsert\_source`,

`upsert\_channel`, `\_row\_to\_source`, the `Source` model, and how `channel\_id` / titles are handled in the handler.

10. user (2026-07-06T09:07:37.541Z)

```
99:     watched                INTEGER NOT NULL DEFAULT 0,
522:     # 1a. Backfill sources from channels first (known title + watched=1). Order
523:     #     matters: channels-first so title/watched data wins; the videos-based
528:         (account_id, provider, provider_source_id, title, kind, watched,
539:     #     ? the channels-based row above survives with its title + watched=1.
543:         (account_id, provider, provider_source_id, kind, watched, seen_first_at)
843: def _row_to_source(row:aiosqlite.Row) -> Source:
851:     watched=bool(row["watched"]),
1104: async def upsert_source(conn:aiosqlite.Connection, source: Source) -> Source:
1105:     """Insert a source row, or update title/watched on conflict.
1112:         (account_id, provider, provider_source_id, title, kind, watched,
1117:         watched=excluded.watched,
1125:         int(source.watched),
1142: async def get_source_by_provider_id(
1387: async def upsert_channel(conn:aiosqlite.Connection, channel: Channel) -> Channel:
1402:     watched=True, # back-compat: pre-ADR-0011 channels were all watched
1537: async def set_source_watched(
1538:     conn:aiosqlite.Connection, account_id: int, source_id: int, watched: bool
1540:     """Toggle the `watched` flag on a source (ADR 0009, Track E)."""
1542:     "UPDATE sources SET watched = ? WHERE id = ? AND account_id = ?",
1543:     (int(watched), source_id, account_id),
1549:     conn:aiosqlite.Connection, account_id: int, *, watched_only: bool = False
1551:     """List all sources for an account, optionally filtering to watched only."""
1555:     if watched_only:
1556:         query += " AND watched = 1"
1557:     query += " ORDER BY watched DESC, provider_source_id ASC"
1578:     watched=True,
1746:     "set_source_watched",
```

11. user (2026-07-06T09:07:38.144Z)

```
109:     title: str | None = None
123: class Source(BaseModel):
127:     providers add their own source kinds). ``provider_source_id`` is the
136:     provider_source_id: str = Field(
139:     title: str | None = None
141:     watched: bool = Field(
237:     title: str | None = None
```

12. user (2026-07-06T09:07:41.013Z)

```
1142     async def get_source_by_provider_id(
1143         conn:aiosqlite.Connection, account_id: int, provider: str, provider_source_id: str
1144     ) -> Source | None:
1145         """Resolve a source by its provider-native id."""
1146         conn.row_factory =aiosqlite.Row
1147         async with conn.execute(
1148             """SELECT * FROM sources
1149                 WHERE account_id = ? AND provider = ? AND provider_source_id = ?""",
1150             (account_id, provider, provider_source_id),
1151         ) as cur:
1152             row = await cur.fetchone()
1153             return _row_to_source(row) if row else None
1154
1155
1156 # --- ContentItem repos (ADR 0011) -----
1157
1158
1159 async def insert_content_item(conn:aiosqlite.Connection, item: ContentItem) ->
```

```

ContentItem:
1160     """Insert a content item. Raises ``IntegrityError`` on a dedup collision.
1161
1162     The ``dedup_key`` must be precomputed by :func:`compute_dedup_key`. Two
1163     unique constraints guard against duplicates:
1164     - ``UNIQUE (account_id, source_id, provider_message_id)`` ? Tier-0.
1165     - ``UNIQUE (account_id, dedup_key)`` ? Tier-1/2 (provider-neutral).
1166     """
1167     cur = await conn.execute(
1168         """INSERT INTO content_items
1169             (account_id, source_id, provider, provider_message_id, content_type,
1170              text, link_url, media_ref, document_id, duration_s, width, height,
1171              size_bytes, mime_type, dedup_key, matched, seen_at)

```

13. user (2026-07-06T09:07:42.090Z)

```

150     engine_holder: dict[str, FilterEngine],
151     client: TelegramClient,
152     delivery_mode: DeliveryMode = DeliveryMode.FORWARD,
153     delivery_target: str = "me",
154 ) -> _NewMessageCallback:
155     """Return an async callback for ``events.NewMessage``.
156
157     The callback:
158     1. extracts VideoMeta + document_id (skips if not a video),
159     2. dedups via two-tier content dedup (document.id, then metadata fingerprint),
160     plus the existing (channel, message) instance dedup,
161     3. runs the filter engine (read from ``engine_holder`` so config
162     changes via ``/set`` take effect immediately),
163     4. inserts a VideoRecord row,
164     5. delivers a match to ``delivery_target`` (see ADR 0005).
165     """
166
167     async def on_new_message(event: NewMessage.Event) -> None: # pragma: no cover
168         message: Message = event.message # type: ignore[assignment]
169         if message is None:
170             return # Telethon edge case: event has no message (shouldn't happen)
171         # Only process genuine channel/group posts and private messages we
172         # receive ? outgoing messages (sent by our own account) are ignored.
173         if getattr(message, "out", False):
174             return
175
176         meta = extract_video_meta(message)
177         if meta is None:
178             return # not a video
179
180         channel_id = _peer_id(message.peer_id)
181         message_id = message.id
182         document_id = _extract_document_id(message)
183
184         # --- watched gate (ADR 0009, Track E) -----
185         # Skip ingestion for unwatched sources. The channel must already be in
186         # sources (lazy-created by the /channels enumeration or the legacy
187         # upsert_channel below); if absent, we let the legacy path upsert it
188         # with watched=True for backwards-compatibility.
189         src = await db.get_source_by_provider_id(conn, user.id, "telegram",
190 str(channel_id))
191         if src is not None and not src.watched:
192             return # silently skip unwatched source
193
194         # --- three-tier dedup (cheapest first) ---
195
196         # Tier 0: same (user, channel, message) instance ? already processed.
197         existing = await db.get_video(conn, user.id, channel_id, message_id)
198         if existing is not None:

```

```

198         return
199
200     # Tier 1: same Telegram document.id ? forwarded/cross-posted file.
201     if document_id is not None:
202         existing = await db.get_video_by_document_id(conn, user.id, document_id)
203         if existing is not None:
204             log.info(
205                 "video_dedup_document_id user=%s doc_id=%s channel=%s msg=%s "
206                 "original_channel=%s original_msg=%s",
207                 user.id,
208                 document_id,
209                 channel_id,
210                 message_id,
211                 existing.channel_id,
212                 existing.message_id,
213             )
214             return
215
216     # Tier 2: metadata fingerprint ? same file re-uploaded (rare but possible).
217     if (
218         meta.size_bytes is not None
219         and meta.duration_s is not None
220         and meta.height is not None
221         and meta.width is not None
222     ):
223         existing = await db.get_video_by_content_fingerprint(
224             conn,
225             user.id,
226             meta.size_bytes,
227             meta.duration_s,
228             meta.height,
229             meta.width,

```

14. assistant (2026-07-06T09:07:46.404Z)

Now I understand the data model. Let me verify the core claims with a repro. Let me check how upsert_channel/upsert_source populate title and provider_source_id, and how backfill discovers channels.

15. user (2026-07-06T09:07:47.915Z)

```

1387     async def upsert_channel(conn: aiosqlite.Connection, channel: Channel) -> Channel:
1388         """Deprecated shim: upsert the :class:`Source` implied by a legacy Channel.
1389
1390         ADR 0011: ``channels`` is renamed to ``channels_deprecated``; this function
1391         reads/writes ``sources`` instead and maps the legacy ``Channel`` shape back.
1392         New code should call :func:`upsert_source` directly.
1393         """
1394         src = await upsert_source(
1395             conn,
1396             Source(
1397                 account_id=channel.user_id,
1398                 provider="telegram",
1399                 provider_source_id=str(channel.channel_id),
1400                 title=channel.title,
1401                 kind="channel",
1402                 watched=True, # back-compat: pre-ADR-0011 channels were all watched
1403                 seen_first_at=channel.seen_first_at,
1404             ),
1405         )
1406         return _source_to_channel(src)
1407
1408
1409     async def get_channel(conn: aiosqlite.Connection, user_id: int, channel_id: int) ->
Channel | None:

```

```

1410     """Deprecated shim: project the :class:`Source` row back to a legacy Channel."""
1411     src = await get_source_by_provider_id(conn, user_id, "telegram", str(channel_id))
1412     return _source_to_channel(src) if src else None
1413
1414
1415     def _source_to_channel(src: Source) -> Channel:
1416         """Project a provider-neutral :class:`Source` onto the legacy :class:`Channel`.
```

16. user (2026-07-06T09:07:50.189Z)

```

src/tg_video_filter/command_service.py:154:         "`/backfill [N][hdw]` ? Scan last N
hours/days/weeks of channel history (default 90d)",
src/tg_video_filter/command_service.py:155:         "`/stop` ? Cancel running backfill",
src/tg_video_filter/command_service.py:377:         "channel. Run `/backfill` to discover
historical channels, "
---channels cmd anywhere?---
src/tg_video_filter/backfill.py:120:     dialogs = await client.get_dialogs()
src/tg_video_filter/backfill.py:123:     for dialog in dialogs:
src/tg_video_filter/command_service.py:45: _CHANNELS_RE = re.compile(r"^/channels$",
re.IGNORECASE)
src/tg_video_filter/command_service.py:326:         if _CHANNELS_RE.match(text):
src/tg_video_filter/commands.py:31:     _CHANNELS_RE,
```

17. assistant (2026-07-06T09:07:54.101Z)

Confirmed: `/channels` handler only calls `db.list\_account\_sources()` ? it does NOT call `client.get\_dialogs()`. In fact `CommandService` has no telegram client at all. This is a clear ADR deviation. Now let me run the repro to confirm the `\_resolve\_source` claims with a realistic source.

18. user (2026-07-06T09:07:55.825Z)

```

---upsert_source signature/title handling---
async def upsert_source(conn: aiosqlite.Connection, source: Source) -> Source:
    """Insert a source row, or update title/watched on conflict.

    The conflict key is `(account_id, provider, provider_source_id)` per the
    ADR 0011 schema. Returns the persisted row.
    """
    await conn.execute(
        """INSERT INTO sources
        (account_id, provider, provider_source_id, title, kind, watched,
        seen_first_at, last_seen_at)
        VALUES (?, ?, ?, ?, ?, ?, ?, ?)
        ON CONFLICT(account_id, provider, provider_source_id) DO UPDATE SET
        title=excluded.title,
        watched=excluded.watched,
        last_seen_at=excluded.last_seen_at""",
        (
            source.account_id,
            source.provider,
            source.provider_source_id,
            source.title,
            source.kind,
            int(source.watched),
            source.seen_first_at.isoformat(),
            (source.last_seen_at.isoformat() if source.last_seen_at else None),
        ),
    )
    await conn.commit()
    conn.row_factory = aiosqlite.Row
    async with conn.execute(
        """SELECT * FROM sources
        WHERE account_id = ? AND provider = ? AND provider_source_id = ?""",
```

```

        (source.account_id, source.provider, source.provider_source_id),
    ) as cur:
        row = await cur.fetchone()
        assert row is not None # just upserted by the conflict key
        return _row_to_source(row)

```

19. assistant (2026-07-06T09:07:58.072Z)

Now let me write and run the repro to confirm the resolution failures with a realistic source (numeric id, username/title).

20. user (2026-07-06T09:07:59.345Z)

```

src/tg_video_filter/db.py:199:async def _apply_schema(conn: aiosqlite.Connection) -> None:
src/tg_video_filter/db.py:936:async def upsert_account(conn: aiosqlite.Connection, account:
Account) -> Account:
src/tg_video_filter/command_service.py:181:class CommandService:
src/tg_video_filter/command_service.py:201:    async def handle(

```

21. user (2026-07-06T09:08:03.415Z)

```

181     class CommandService:
182         """Provider-independent command execution.
183
184         Parses a command string against an account's current config and DB,
185         returning a ``CommandResult``. No Telethon objects involved ? pure string
186         in, effects out. The Telethon adapter (``commands.py``) handles the
187         transport (checking message.out, sending the reply); the Bot API adapter
188         (future) calls ``handle()`` with the ``from.id`` from the update.
189
190         Usage::
191
192             svc = CommandService()
193             result = await svc.handle(user_id, text, conn, current_cfg)
194             if result.config_changed:
195                 engine_holder["engine"] = FilterEngine(current_cfg)
196             await send_reply(result.reply_text)
197         """
198
199         # -- public API -----
200
201         async def handle(
202             self,
203             user_id: int,
204             text: str,
205             conn: aiosqlite.Connection,
206             current_cfg: FilterConfig,
207         ) -> CommandResult:
208             """Parse and execute a command text, returning the reply + change flag."""
209             text = text.rstrip()
210
211             # --- /set size <value><unit> -----
212             m = _SIZE_RE.match(text)
213             if m:
214                 value_str, unit = m.group(1), m.group(2) or ""
215                 # NOTE: ValueError is unreachable here ? the regex already validates
216                 # the unit against [kmgT]?b?, so parse_size_mb can never raise.
217                 # Kept for defensive parity with the original commands.py refactor.
218                 try:
219                     new_mb = parse_size_mb(value_str, unit)
220                 except ValueError:

```

22. user (2026-07-06T09:08:04.640Z)

```

35:class Account(BaseModel):
123:class Source(BaseModel):
async def upsert_account(conn: aiomysqlite.Connection, account: Account) -> Account:
    """Insert or update an account row; returns the persisted Account."""
    await conn.execute(
        """INSERT INTO accounts (id, telegram_id, provider, display_name, created_at)
        VALUES (?, ?, ?, ?, ?)
        ON CONFLICT(id) DO UPDATE SET
            telegram_id=excluded.telegram_id,
            provider=excluded.provider,
            display_name=excluded.display_name""",
        (
            account.id,
            account.telegram_id,
            account.provider,
            account.display_name,
            account.created_at.isoformat(),
        ),
    )
    await conn.commit()
    return await get_account(conn, account.id) # type: ignore[return-value]

async def get_account(conn: aiomysqlite.Connection, account_id: int) -> Account | None:
    conn.row_factory = aiomysqlite.Row
    async with conn.execute("SELECT * FROM accounts WHERE id = ?", (account_id,)) as cur:
        row = await cur.fetchone()

```

23. user (2026-07-06T09:08:08.180Z)

```

35    class Account(BaseModel):
36        """A product identity ? the owner of config, channels, routes, and content.
37
38        Decoupled from any specific provider's session (ADR 0010). ``telegram_id``
39        is the bridge used by the Bot API front-door (``from.id``) and by the
40        legacy userbot path; it is nullable so future providers that don't have a
41        Telegram id (e.g. WhatsApp) can still own an ``Account``.
42        """
43
44        model_config = ConfigDict(frozen=True)
45
46        id: int = Field(description="Stable surrogate PK; the owner key for all downstream
47        tables")
48        telegram_id: int | None = Field(
49            default=None,
50            description="Telegram user id (bot from.id OR userbot account id). UNIQUE when
51            set.",
52        )
53        provider: str = Field(default="telegram", description="Provider namespace; future:
54        'whatsapp'")
55        display_name: str | None = None
56        created_at: datetime = Field(default_factory=_utcnow)
57
58    class TelegramSession(BaseModel):
59        """An optional MTPROTO (Telethon) session linked to an :class:`Account`.
60
61        A bot-only user has an ``Account`` row but no ``TelegramSession`` row;
62        that is what makes them structurally invisible to ``manager.run``
63        (ADR 0010). ``session_path`` is data-driven so the file-location
64        convention is explicit rather than derived from the PK at runtime.
65        """
66

```

24. user (2026-07-06T09:08:08.796Z)

```

123     class Source(BaseModel):
124         """A configured place we ingest content from (ADR 0011).
125
126         Generalizes the legacy :class:`Channel` (1:1 for Telegram today; future
127         providers add their own source kinds). ``provider_source_id`` is the
128         provider-native id (Telegram ``channel_id``, future WhatsApp group id, ?).
129         """
130
131         model_config = ConfigDict(frozen=True)
132
133         id: int | None = Field(default=None, description="Surrogate PK (DB-assigned)")
134         account_id: int = Field(description="FK ? accounts.id (owner of the source)")
135         provider: str = Field(default="telegram", description="Provider namespace")
136         provider_source_id: str = Field(
137             description="Provider-native source id (Telegram channel_id, ?)"
138         )
139         title: str | None = None
140         kind: str = Field(default="channel", description="'channel' | 'group' | 'feed'")
141         watched: bool = Field(
142             default=False,
143             description="ADR 0009 selective scanning: ingest from this source iff True",
144         )
145         seen_first_at: datetime = Field(default_factory=_utcnow)
146         last_seen_at: datetime | None = None
147
148     class ContentItem(BaseModel):
149         """A normalized content record discovered from any :class:`Source` (ADR 0011).
150
151         Generalizes the legacy :class:`VideoRecord`. Phase 1 stores only videos

```

25. assistant (2026-07-06T09:08:11.011Z)

Now I have everything needed. Let me run the repro with a realistic source.

26. user (2026-07-06T09:10:17.838Z)

Exit code 143
Command timed out after 2m 0s

27. assistant (2026-07-06T09:10:22.458Z)

The ``.venv`` python path may be the issue with the bash path. Let me use PowerShell.

28. user (2026-07-06T09:11:24.719Z)

Command running in background with ID: b6bzc84dj. Output is being written to:
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\tasks\b6bzc84dj.output. You will be notified when it completes. To check interim output, use Read on that file path.

29. user (2026-07-06T09:11:28.974Z)

```

1     Traceback (most recent call last):
2         File "C:\Users\avidu\AppData\Local\Temp\claude\repro.py", line 31, in <module>
3             asyncio.run(main())
4             ~~~~~^~~~~~
5         File "C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\asyncio\runners.py",
line 195, in run
6             return runner.run(main)
7             ~~~~~^~~~~~
8         File "C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\asyncio\runners.py",
line 118, in run
9             return self._loop.run_until_complete(task)

```

```

10             ~~~~~^
11         File
12         "C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\asyncio\base_events.py", line 725,
13         in run_until_complete
14             return future.result()
15         ~~~~~^
16         File "C:\Users\avidu\AppData\Local\Temp\claude\repro.py", line 8, in main
17             acc = await db.upsert_account(conn, Account(provider="telegram",
18             provider_account_id="u1", label="t"))
19         ~~~~~^
20         ~~~~~^
21         File "C:\Users\avidu\Projects\Telegram\.venv\Lib\site-packages\pydantic\main.py", line
22         263, in __init__
23             validated_self = self.__pydantic_validator__.validate_python(data,
24             self_instance=self)
25         pydantic_core._pydantic_core.ValidationError: 1 validation error for Account
26         id
27         Field required [type=missing, input_value={'provider': 'telegram', ...id': 'u1',
28         'label': 't'}, input_type=dict]
29         For further information visit https://errors.pydantic.dev/2.13/v/missing
30

```

30. assistant (2026-07-06T09:11:32.838Z)

The heredoc didn't overwrite the file (bash `cat` failed earlier ? the old file remained). Let me write the repro via the Write tool.

31. assistant (2026-07-06T09:11:38.196Z)

```

import asyncio, aiosqlite
from tg_video_filter import db
from tg_video_filter.models import Account, Source, FilterConfig
from tg_video_filter.command_service import CommandService

async def main():
    conn = await aiosqlite.connect(":memory:")
    await db._apply_schema(conn)
    await db.upsert_account(conn, Account(id=1, telegram_id=555))
    await db.upsert_source(conn, Source(
        account_id=1, provider="telegram",
        provider_source_id="-1001234567890",
        title="Classical Music Daily",
        kind="channel", watched=False,
    ))
    svc = CommandService()
    cfg = FilterConfig()

    async def run(text):
        r = await svc.handle(1, text, conn, cfg)
        print(f" {text!r:42} -> {r.reply_text!r}")

    print("Realistic source: id='-1001234567890', title='Classical Music Daily',
watched=False\n")
    print("/watch by @username:")
    await run("/watch @classicalmusic")
    print("/watch by display name/title:")
    await run("/watch Classical Music Daily")
    print("/watch by numeric id:")
    await run("/watch -1001234567890")
    print("lstrip multiple @ then numeric id:")
    await run("/watch @@@-1001234567890")
    print("/channels (does it enumerate get_dialogs candidates?):")
    await run("/channels")
    await conn.close()

```



```
asyncio.run(main())
```

32. user (2026-07-06T09:11:38.197Z)

<tool_use_error>File has not been read yet. Read it first before writing to it.</tool_use_error>

33. assistant (2026-07-06T09:11:44.495Z)

```
import asyncio, aiosqlite
from tg_video_filter import db
from tg_video_filter.models import Account, Source, FilterConfig
from tg_video_filter.command_service import CommandService

async def main():
    conn = await aiosqlite.connect(":memory:")
    await db._apply_schema(conn)
    await db.upsert_account(conn, Account(id=1, telegram_id=555))
    await db.upsert_source(conn, Source(
        account_id=1, provider="telegram",
        provider_source_id="-1001234567890",
        title="Classical Music Daily",
        kind="channel", watched=False,
    ))
    svc = CommandService()
    cfg = FilterConfig()

    async def run(text):
        r = await svc.handle(1, text, conn, cfg)
        print(f" {text!r:42} -> {r.reply_text!r}")

    print("Realistic source: id='-1001234567890', title='Classical Music Daily',
watched=False\n")
    print("/watch by @username:")
    await run("/watch @classicalmusic")
    print("/watch by display name/title:")
    await run("/watch Classical Music Daily")
    print("/watch by numeric id:")
    await run("/watch -1001234567890")
    print("rstrip multiple @ then numeric id:")
    await run("/watch @@@-1001234567890")
    print("/channels (does it enumerate get_dialogs candidates?):")
    await run("/channels")
    await conn.close()

asyncio.run(main())
```

34. user (2026-07-06T09:11:44.499Z)

<tool_use_error>File has not been read yet. Read it first before writing to it.</tool_use_error>

35. assistant (2026-07-06T09:11:48.837Z)

A file with that name apparently exists. Let me use a fresh unique name.

36. assistant (2026-07-06T09:11:53.709Z)

```
import asyncio, aiosqlite
from tg_video_filter import db
from tg_video_filter.models import Account, Source, FilterConfig
from tg_video_filter.command_service import CommandService

async def main():
    conn = await aiosqlite.connect(":memory:")
    await db._apply_schema(conn)
```

```

await db.upsert_account(conn, Account(id=1, telegram_id=555))
await db.upsert_source(conn, Source(
    account_id=1, provider="telegram",
    provider_source_id="-1001234567890",
    title="Classical Music Daily",
    kind="channel", watched=False,
))
svc = CommandService()
cfg = FilterConfig()

async def run(text):
    r = await svc.handle(1, text, conn, cfg)
    print(f" {text!r:42} -> {r.reply_text!r}")

    print("Realistic source: id='-1001234567890', title='Classical Music Daily',
watched=False\n")
    print("/watch by @username:")
    await run("/watch @classicalmusic")
    print("/watch by display name/title:")
    await run("/watch Classical Music Daily")
    print("/watch by numeric id:")
    await run("/watch -1001234567890")
    print("lstrip multiple @ then numeric id:")
    await run("/watch @@@-1001234567890")
    print("/channels (does it enumerate get_dialogs candidates?):")
    await run("/channels")
    await conn.close()

asyncio.run(main())

```

37. user (2026-07-06T09:11:53.737Z)

File created successfully at: C:\Users\avidu\AppData\Local\Temp\claude\repro_track_e_v3.py (file state is current in your context ? no need to Read it back)

38. user (2026-07-06T09:12:00.642Z)

Realistic source: id='-1001234567890', title='Classical Music Daily', watched=False

```

/watch by @username:
  '/watch @classicalmusic'          -> "? Source `@classicalmusic` not found. Use
`/channels` to list available sources.\n\nSources are discovered as messages arrive; a channel
you haven't received traffic from may not appear yet."
/watch by display name/title:
  '/watch Classical Music Daily'      -> "? Source `Classical Music Daily` not found. Use
`/channels` to list available sources.\n\nSources are discovered as messages arrive; a channel
you haven't received traffic from may not appear yet."
/watch by numeric id:
  '/watch -1001234567890'            -> '? Now watching **Classical Music Daily**
(`-1001234567890`).'
lstrip multiple @ then numeric id:
  '/watch @@@-1001234567890'        -> '? Now watching **Classical Music Daily**
(`-1001234567890`).'
/channels (does it enumerate get_dialogs candidates?):
  '/channels'                        -> '? **All sources**\n\n? `-1001234567890` ?
Classical Music Daily'

```

39. assistant (2026-07-06T09:12:05.209Z)

Confirmed all claims. Both `/watch @username` and `/watch <name>` fail for a realistic source; only the numeric id works. ADR line 64 says `/watch <#name|id>` ? resolution by name/username is a contract requirement that is unimplemented.

Let me verify the `/channels` get_dialogs deviation is real: CommandService has no client, and

there's no dialogs upsert anywhere in the command path. Already confirmed via grep ?
`\_handle\_channels` only calls `list\_account\_sources`. Let me also confirm the test that "passes" uses an unrealistic seed.

40. user (2026-07-06T09:12:06.540Z)

```
245:     provider_source_id: str,
254:         provider_source_id=provider_source_id,
262:async def test_channels_lists_all_sources_with_watch_status(
274:async def test_channels_empty_shows_help(conn: aiomysqlite.Connection, svc: CommandService) ->
None:
279:async def test_watchlist_only_watched(conn: aiomysqlite.Connection, svc: CommandService) ->
None:
287:async def test_watchlist_empty_shows_help(conn: aiomysqlite.Connection, svc: CommandService)
-> None:
292:async def test_watch_by_numeric_id(conn: aiomysqlite.Connection, svc: CommandService) -> None:
301:async def test_watch_by_username(conn: aiomysqlite.Connection, svc: CommandService) -> None:
302:     await _seed_source(conn, USER_ID, "@mychan", "My Channel", watched=False)
303:     result = await svc.handle(USER_ID, "/watch @mychan", conn, FilterConfig())
305:     src = await db.get_source_by_provider_id(conn, USER_ID, "telegram", "@mychan")
310:async def test_unwatch(conn: aiomysqlite.Connection, svc: CommandService) -> None:
319:async def test_watch_unknown_source_shows_error(
326:async def test_set_source_watched_toggle(conn: aiomysqlite.Connection) -> None:
335:async def test_list_account_sources_watched_only(
344:     assert watched[0].provider_source_id == "-10"
```

41. assistant (2026-07-06T09:12:14.963Z)

Confirmed: `\_test\_watch\_by\_username` (line 301-303) seeds a source with
`provider\_source\_id="@mychan"` ? an unrealistic id. Real telegram sources never have a
`@username` as their `provider\_source\_id` (it is always `str(channel\_id)`, numeric). So the test
is green only because of the fictitious seed; the feature is non-functional for real sources.

Now let me confirm the `lstrip('@)` multiple-strip is a real (minor) issue and check whether
`set\_source\_watched` silently no-ops on wrong account (correctness of the WHERE clause). These
are correct. Let me also double check the `/channels` "no client" story ? is there any other
layer that upserts dialogs before calling handle? Check commands.py adapter.

42. user (2026-07-06T09:12:16.591Z)

```
src/tg_video_filter/commands.py:1: """Config commands ? Telethon adapter over the provider-
independent `CommandService`.
src/tg_video_filter/commands.py:5: parsing and config mutation to
`command_service.CommandService` (ADR 0011, Track D).
src/tg_video_filter/commands.py:24: from tg_video_filter.command_service import ENGINE_KEY,
CommandResult, CommandService
src/tg_video_filter/commands.py:58:     svc = CommandService()
src/tg_video_filter/commands.py:79:     result: CommandResult = await svc.handle(user.id,
text, conn, current_cfg)
src/tg_video_filter/commands.py:105:     """Deprecated wrapper ? delegates to
`CommandService._update`.
src/tg_video_filter/commands.py:109:     :meth:`CommandService.handle` which returns a
:class:`CommandResult`.
src/tg_video_filter/commands.py:111:     svc = CommandService()
```

43. assistant (2026-07-06T09:12:24.762Z)

Confirmed: no `get\_dialogs` anywhere near the command path. `/channels` is purely a DB list of
already-seen sources. The ADR (lines 48-54) explicitly requires `/channels` to upsert candidate
rows from `client.get\_dialogs()` so users can browse and watch channels they have never received
traffic from. This is unimplemented ? and it's the ADR's stated linchpin for discoverability (it
calls the flag default and enumeration flow "both required").

This has a compounding effect with the opt-in default: a freshly-onboarded user scans nothing

(correct per ADR), but `/channels` shows "No sources yet" and there is no way to discover a channel to `/watch` until traffic happens to arrive ? and even then, only the numeric id works. The discoverability path the ADR designed is absent.

Let me verify one more edge: does `/watch <name>` at least have a fallback that a title match would resolve? No ? `_resolve_source` only queries `get_source_by_provider_id`. Confirmed. I have enough to report.

44. user (2026-07-06T09:12:51.459Z)

Structured output provided successfully